Performance testing Kubernetes-Workloads

CLOUD NATIVE FESTIVAL
im Heide Park Soltau

STEPHAN SCHWARZ

Senior DevOps Consultant
iits-consulting GmbH

20.05.2026
17:00 Uhr

CloudLand
www.cloudland.org

about me

Stephan Schwarz
44 Jahre alt - verheiratet, zwei Kinder

Rollen in der Vergangenheit:

- Network Administrator
- Systems Engineer
- Cloud Engineer
- Infrastructure Developer
- Platform Engineer

seit 2017: **Kubernetes**

seit 2021: **Consultant**

what about you?

Es war einmal vor langer Zeit in einer weit, weit entfernten Galaxis…

Was ist Performance?

Performance = **Erwartungen** werden **erfüllt** an:

- Durchsatz
- Latenz
- Skalierbarkeit
- Ressourcenverbrauch
- Verfügbarkeit

Erwartungen sind **individuell** unterschiedlich

Warum Lasttests?

- **Stabiler Betrieb**
- Testen der **Skalierbarkeit**
- Sicherstellung von **SLAs**
- Ermittlung der benötigten **Ressourcen**
- Optimierung von **Kosten**
- Aufdecken von **Bottlenecks**

Typische Bottlenecks

Requests / Limits

CPU / RAM

Typische Bottlenecks

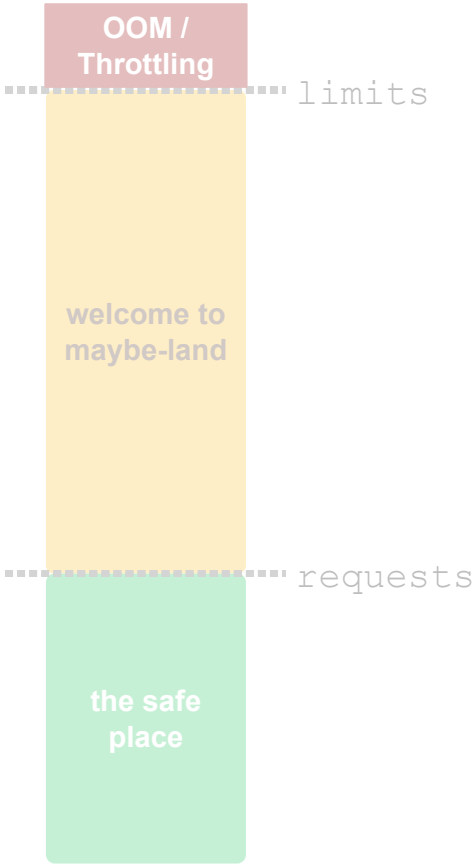
Requests / Limits

CPU / RAM

Typische Bottlenecks

Requests / Limits
CPU / RAM

Disk I/O & Filehandles
IOPS, max. number of file handles and threads



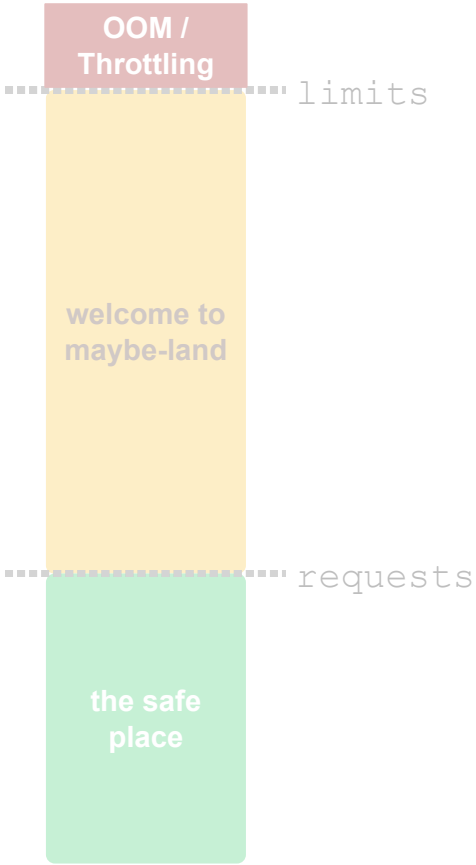
Typische Bottlenecks

Requests / Limits
CPU / RAM

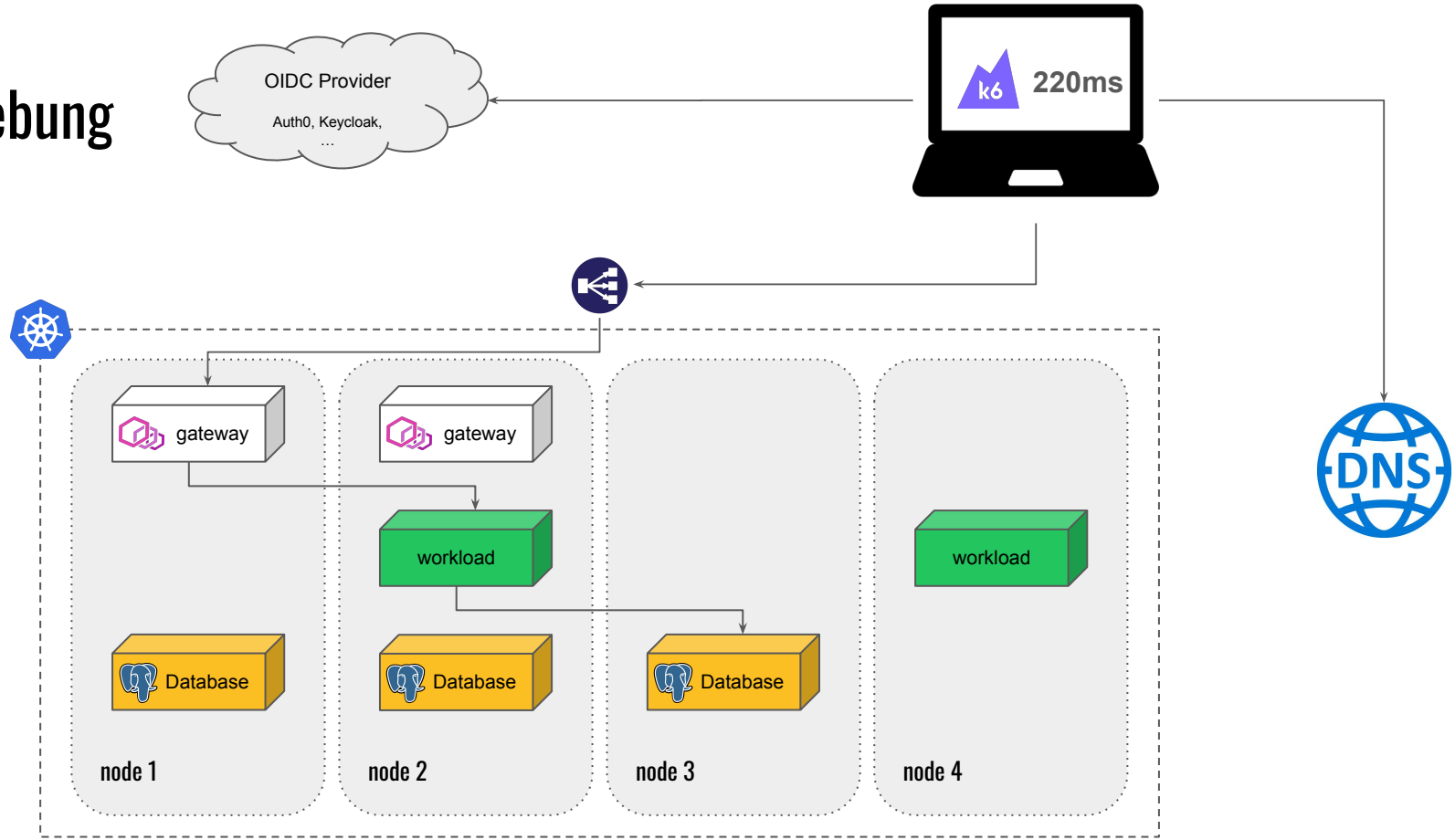
Disk I/O & Filehandles
IOPS, max. number of file handles and threads

Netzwerk
DNS, Ingress Controller, Service Mesh

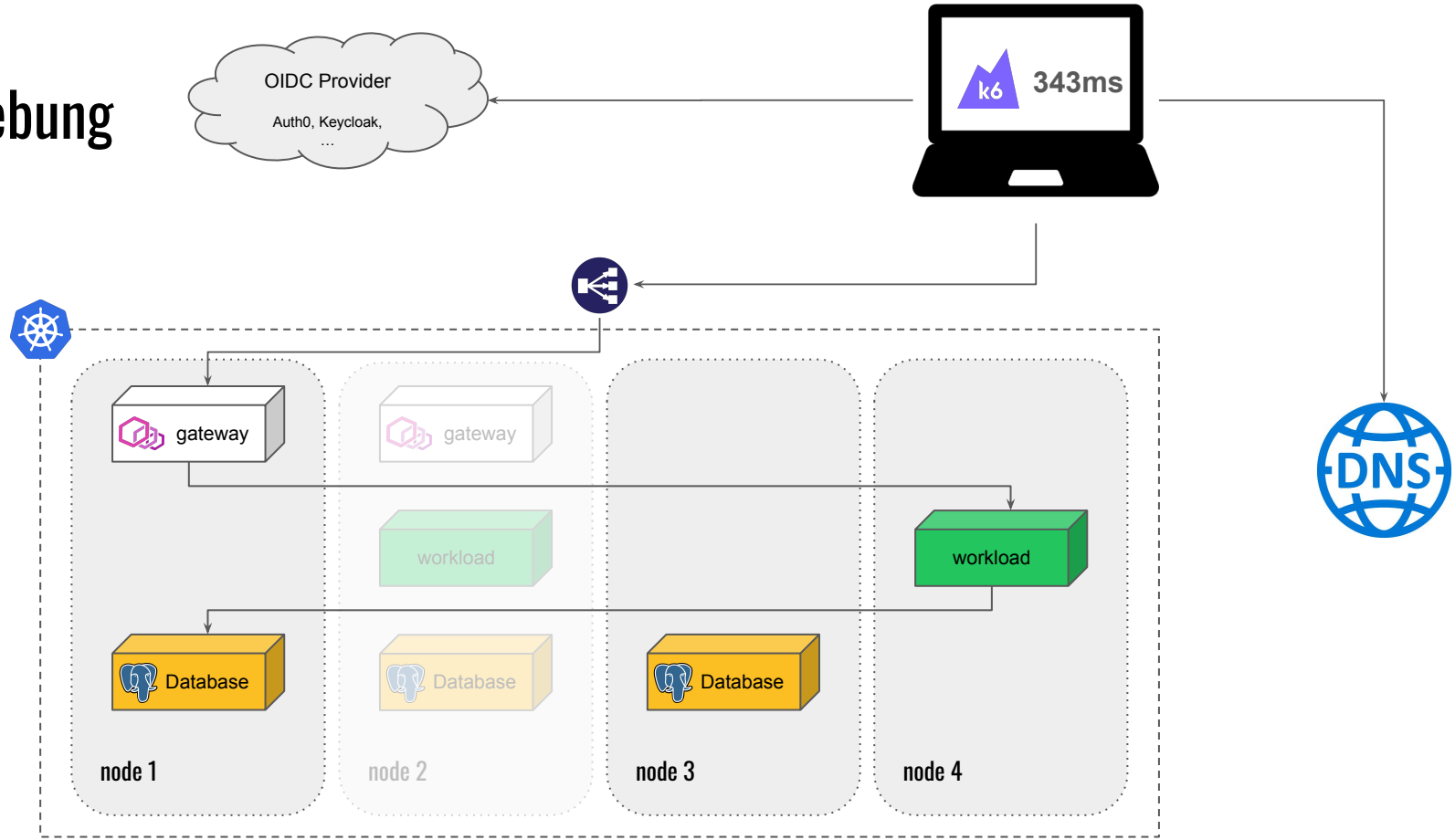
Externe Abhängigkeiten (DB, S3, APIs, ...)



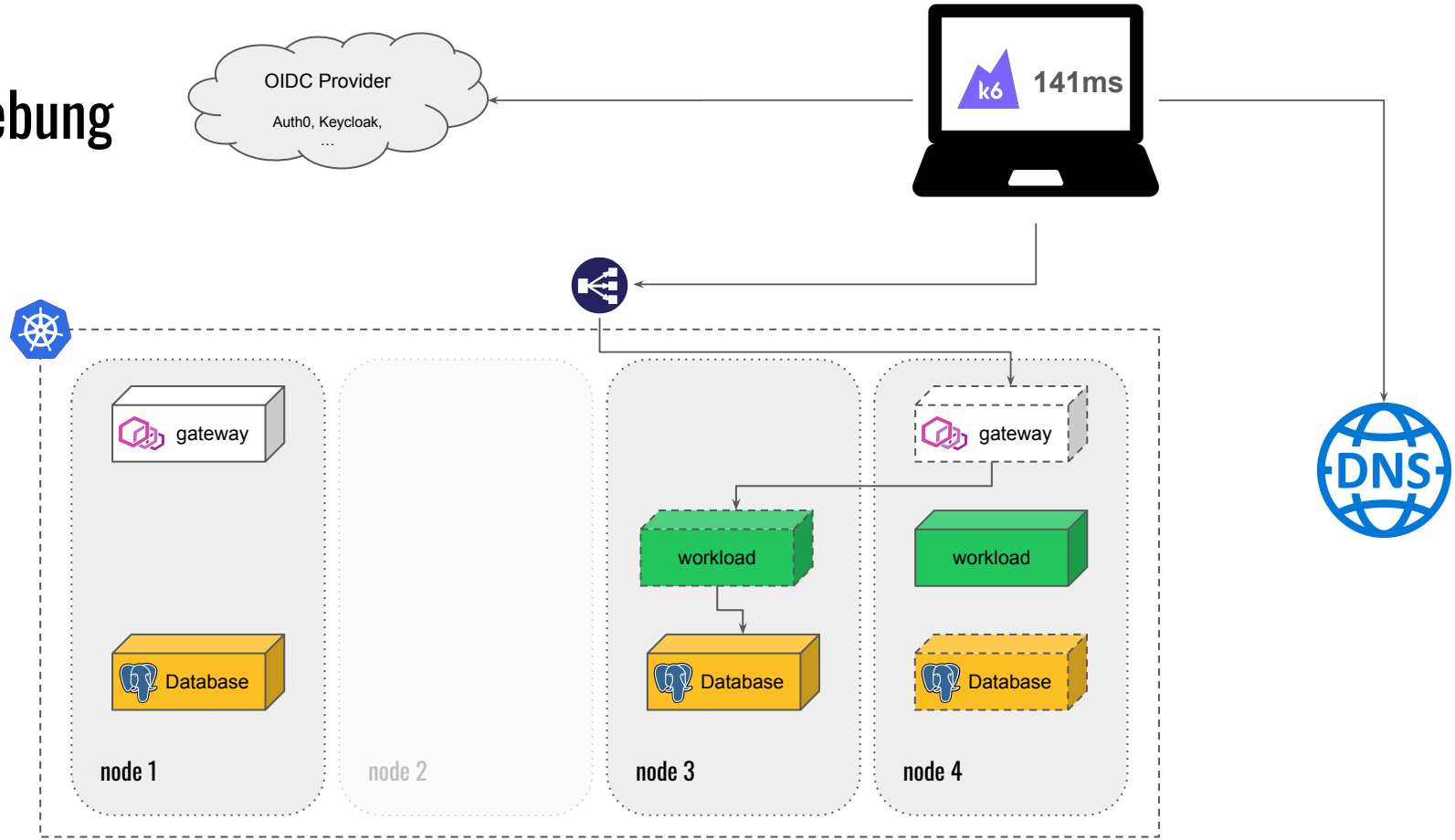
Testumgebung



Testumgebung

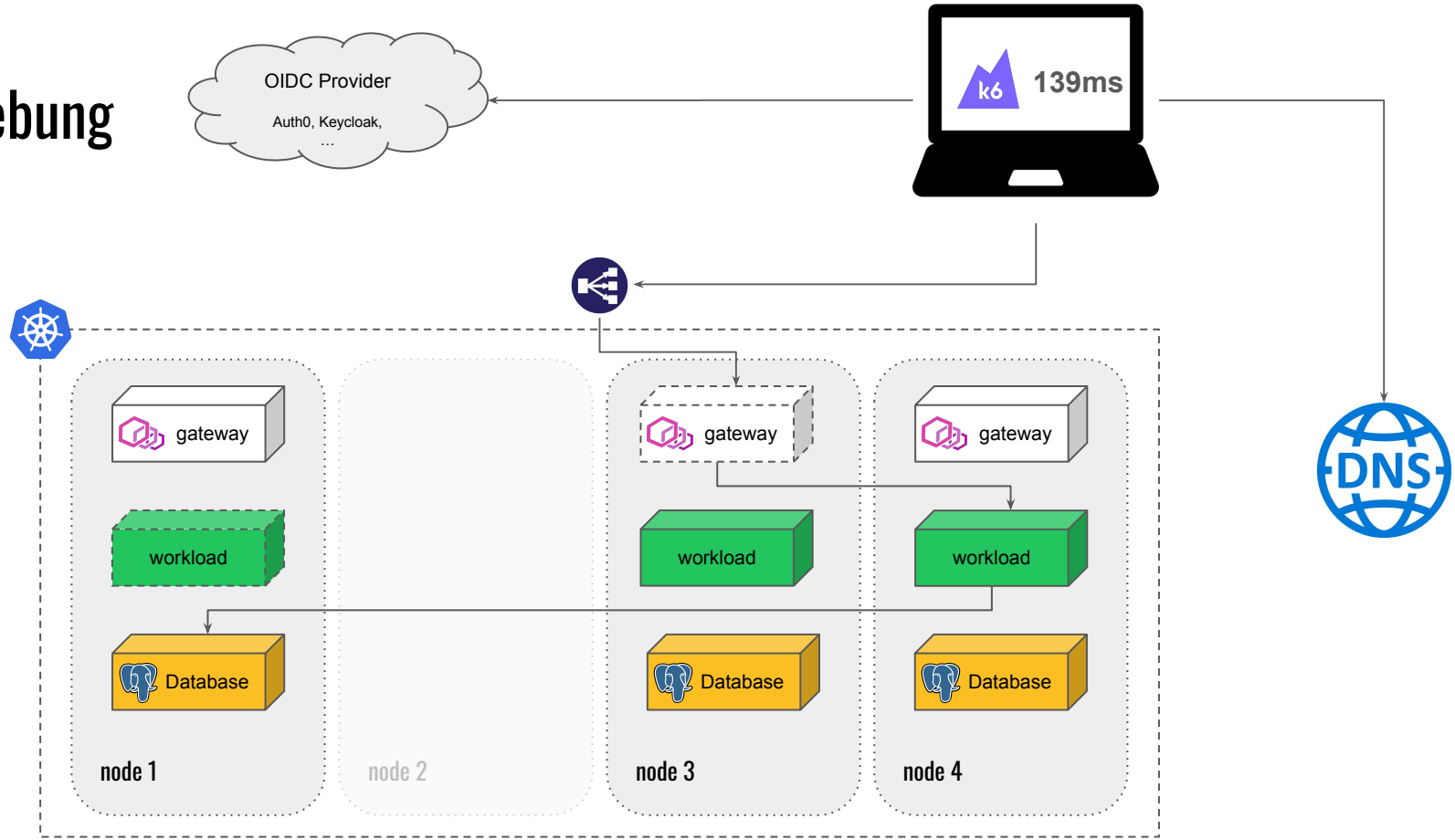


Testumgebung



Re-scheduling

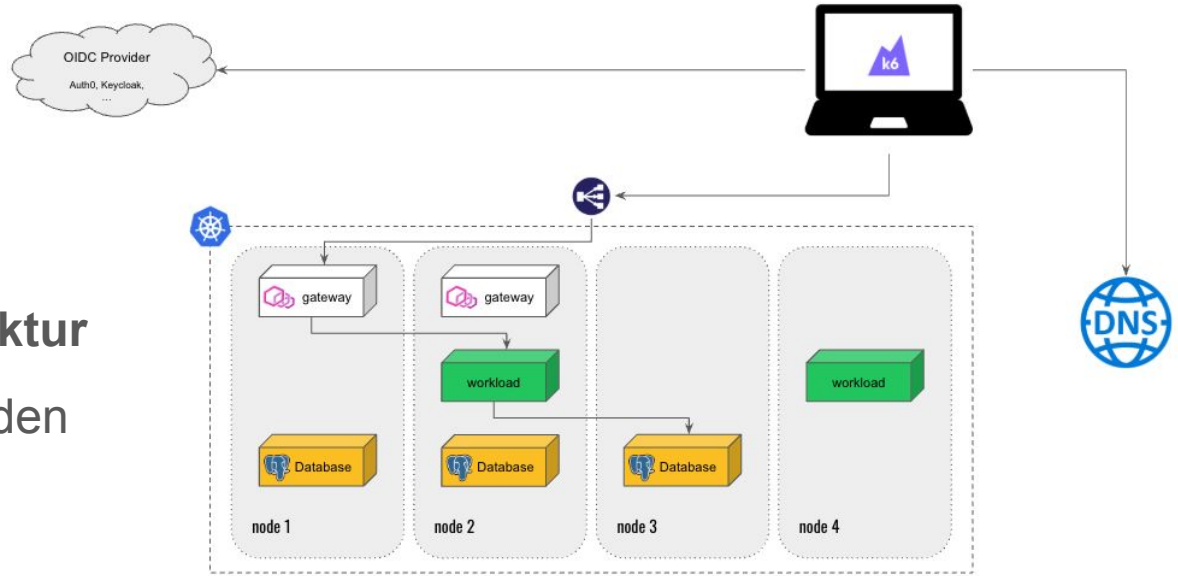
Testumgebung



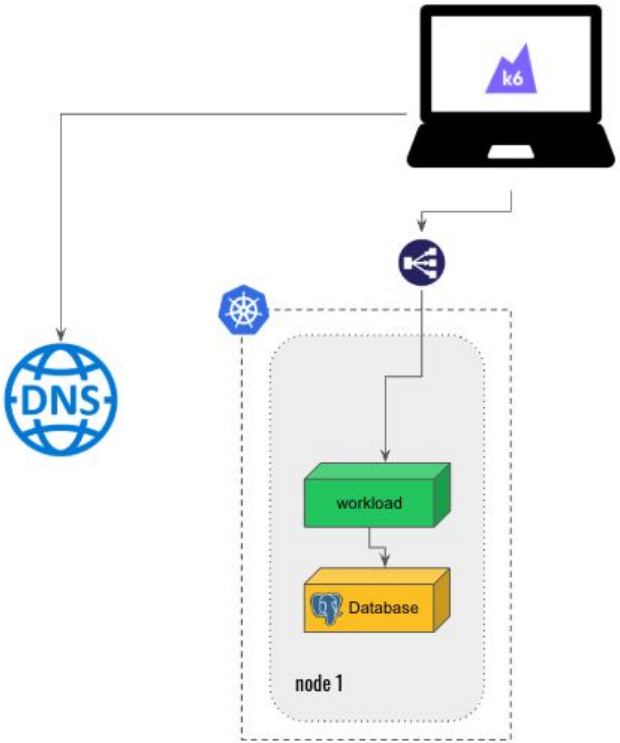
Testumgebung

End-to-end

- **Realitätsnah**
- **Kundenperspektive**
- Testet komplette **Infrastruktur**
- **Abhängige Services** werden mitgetestet



Testumgebung



Labor Umgebung

- z.B. **Single Pod** Perspektive
- Ermittlung der **Resource Usage**
- Experimentieren mit Werten für **requests** und **limits**
- Nur **notwendige Abhängigkeiten** werden mit getestet
- **Single node** cluster ohne autoscaling

Arten von Lasttests

Smoke Test

Genereller Funktionstest

Z.B. zum Validieren eines Test-Szenarios

Average Load Test

Alltagslast

Ermitteln der Baseline
- CPU Verbrauch
- RAM Verbrauch
- Antwortzeit

Stabiler Betrieb (SLA)

Stress Test

Black Friday Test (1/2)

Test des autoscalings
und der Resilienz

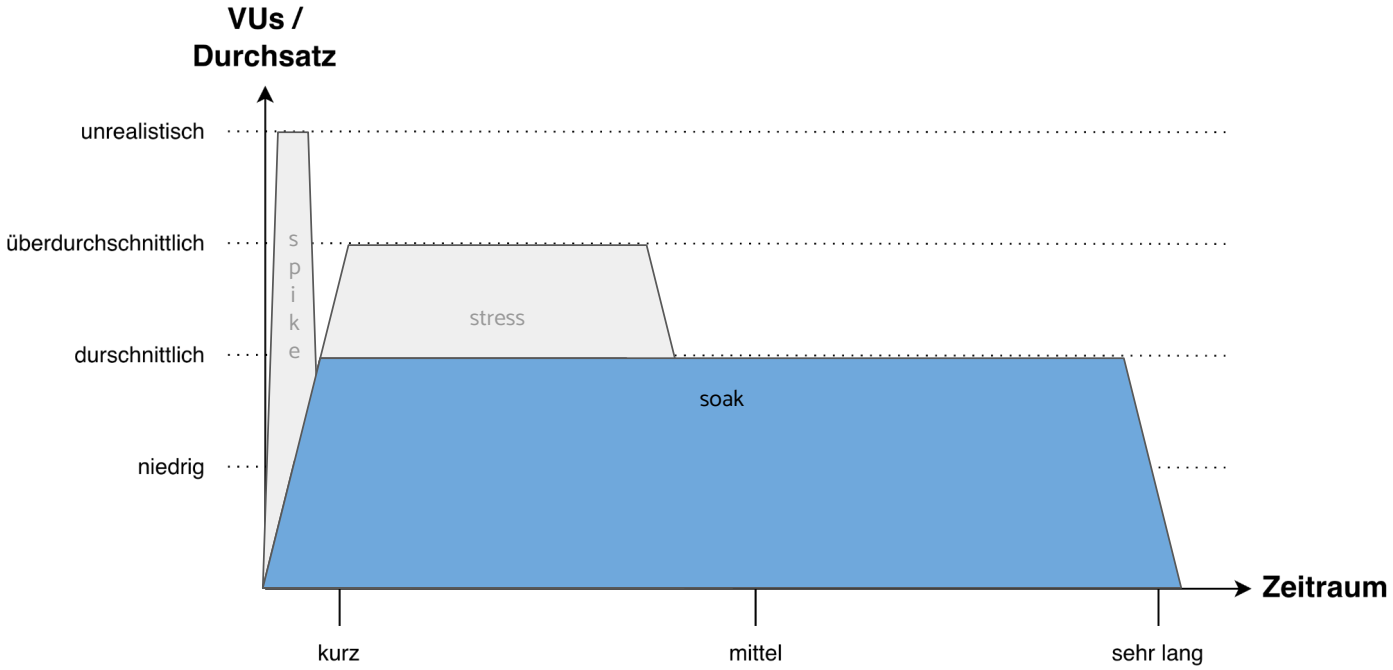
Spike Test

Black Friday Test (2/2)

Scale up/down
- HPA
- Cluster autoscaler

Resilienz
- Ausfall?
- Wie lange? (MTTR)
- Blast Radius

Soak Test



Langzeittest (z.B. 24h)

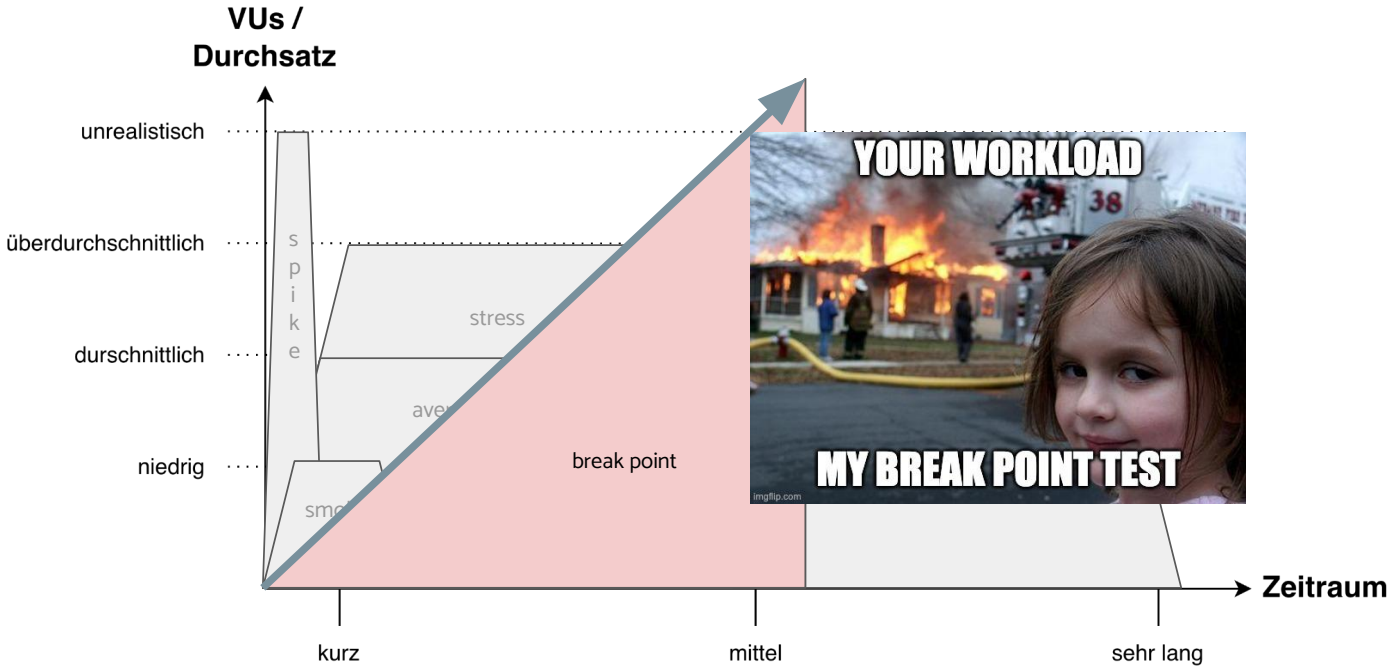
Ermitteln von Langzeit Effekten:

- Memory Leaks
- Connection Pool Exhaustion
- Performance Degradation
- Disks filling up (Logs / Database)

Verhalten während Maintenance Tasks:

- Upgrades
- Migrationen

Break Point Test



Erhöhung bis zum Kollaps

Ermitteln von:

- max. Kapazität
- Bottlenecks

Was ist der richtige Test / die richtige Testumgebung für mich?

It depends…

Welche individuellen Fragen sollen beantwortet werden?

Kombination

Selten kann ein einzelner Test alle relevanten Fragen direkt beantworten

Evolutionär

Im Laufe der Zeit werden sich Ihre Fragen ändern.

Load Generatoren

Anforderungen:

- **Protokoll-Vielfalt:**
  Support für HTTP/S, WebSockets, gRPC, MQTT, TCP, …
- **As-Code & CI/CD-Integration:**
  Definition von Tests als Code (JS, Java, Python) und einfache Einbindung in Pipelines.
- **Verteilte Last (Distributed Testing):**
  Native Fähigkeit, Last über mehrere Worker-Nodes/Cluster hinweg zu skalieren.
- **Flexibles Test-Modeling:**
  Präzise Steuerung von Virtual Users (VU) / Requests/s, Ramp-up/down und komplexen Szenarien. Nutzung von HAR Files
- **Metriken & Echtzeit-Reporting:**
  Export/Streaming von Live-Daten (Prometheus, InfluxDB) und detaillierte HTML-/Konsolen-Berichte.
- **Erweiterbarkeit (Plugins):**
  Nachladen von Drittanbieter-Bibliotheken oder Custom-Protokollen.

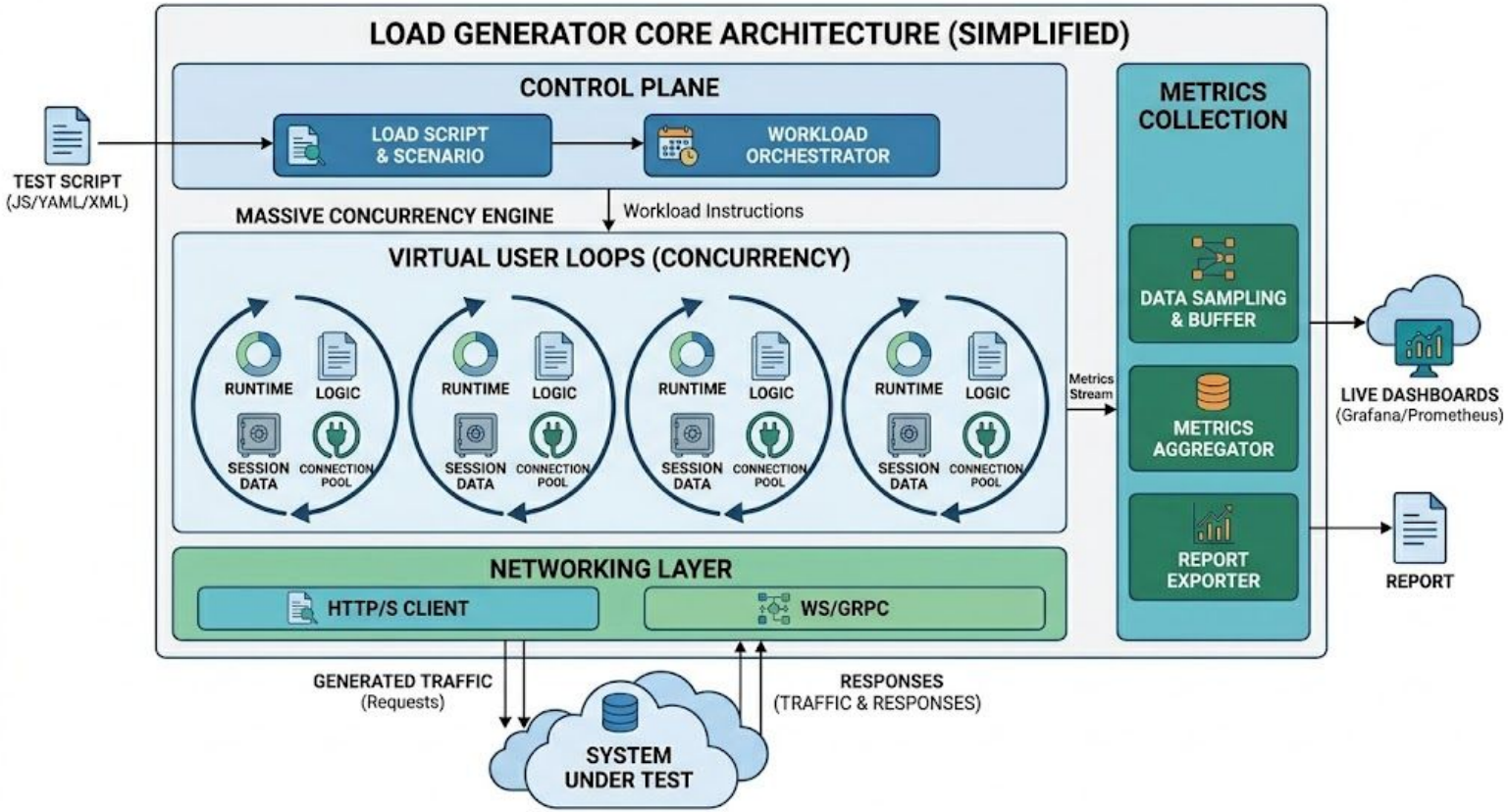
z.B.:

- k6
- Gatling
- Locust
- Artillery
- Apache jMeter

Testbeispiel

```
1  import { check, sleep } from "k6";
2  import http from "k6/http";
3
4  export let options = {
5    stages: [
6      { duration: "1m", target: 5 },    // ramp up
7      { duration: "3m", target: 10 },   // main
8      { duration: "1m", target: 0 },    // ramp down
9    ]
10 };
11
12 export default function() {
13   let res = http.get("https://iits-consulting.de");
14   check(res, {
15     "is status 200": (r) => r.status === 200
16   });
17   sleep(1);
18 };
```

Wie funktioniert ein Load Generator?



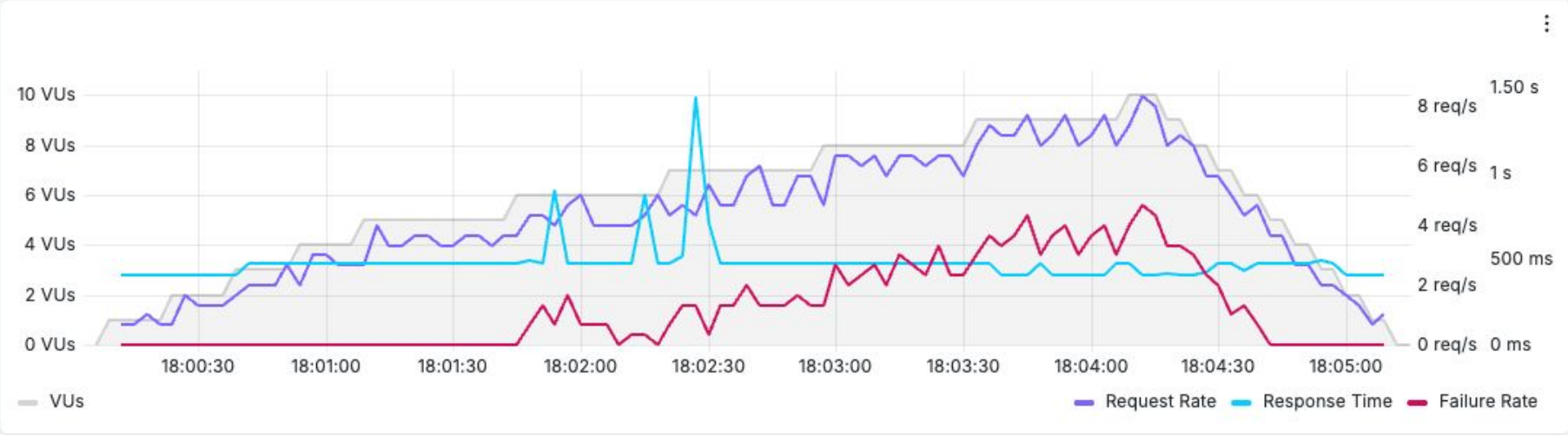

k6 run ./basic-load.js
k6 run ./basic-load.js
Grafana
execution: local
script: ./basic-load.js
output: -
scenarios: (100.00%) 1 scenario, 10 max VUs, 5m30s max duration (incl. graceful stop):
* default: Up to 10 looping VUs for 5m0s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)
running (0m44.5s), 03/10 VUs, 73 complete and 0 interrupted iterations
default [====>----------] 03/10 VUs 0m44.5s/5m00.0s

```
TOTAL RESULTS

  checks_total.......: 1543   5.127954/s
  checks_succeeded...: 71.35% 1101 out of 1543
  checks_failed......: 28.64% 442 out of 1543

  ✗ is status 200
    ↳  71% — ✓ 1101 / ✗ 442

  HTTP
  http_req_duration..............: avg=130.43ms min=22.99ms  med=148.97ms max=1.27s p(90)=180.94ms p(95)=188.84ms
    { expected_response:true }...: avg=165.55ms min=120.53ms med=157.05ms max=1.27s p(90)=184.55ms p(95)=192.75ms
  http_req_failed................: 28.64% 442 out of 1543
  http_reqs......................: 1543   5.127954/s

  EXECUTION
  iteration_duration.............: avg=1.13s    min=1.02s    med=1.15s    max=2.28s p(90)=1.18s    p(95)=1.19s
  iterations.....................: 1543   5.127954/s
  vus............................: 1      min=1           max=10
  vus_max........................: 10     min=10          max=10

  NETWORK
  data_received..................: 203 MB 675 kB/s
  data_sent......................: 771 kB 2.6 kB/s
```

PERFORMANCE OVERVIEW

The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.

REQUESTS MADE	HTTP FAILURES	PEAK RPS	P95 RESPONSE TIME
1.3K reqs	371 reqs	8.33 reqs/s	477 ms



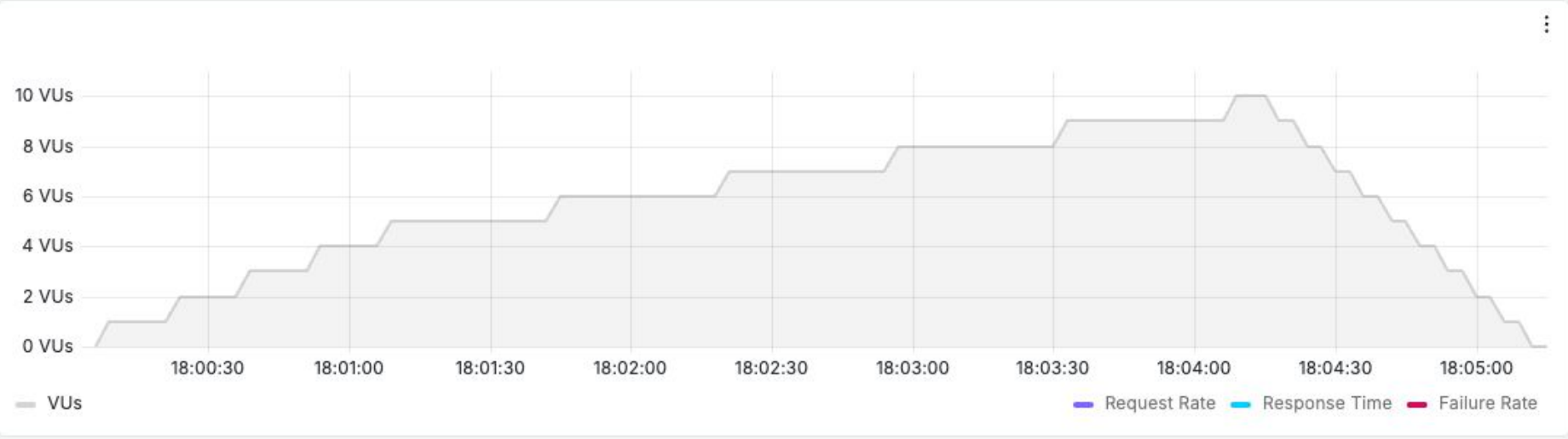
SCENARIOS (0) SHOW

No scenario was configured in this test. Check out how to create one

PERFORMANCE OVERVIEW

The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.

REQUESTS MADE	HTTP FAILURES	PEAK RPS	P95 RESPONSE TIME
1.3K reqs	371 reqs	8.33 reqs/s	477 ms



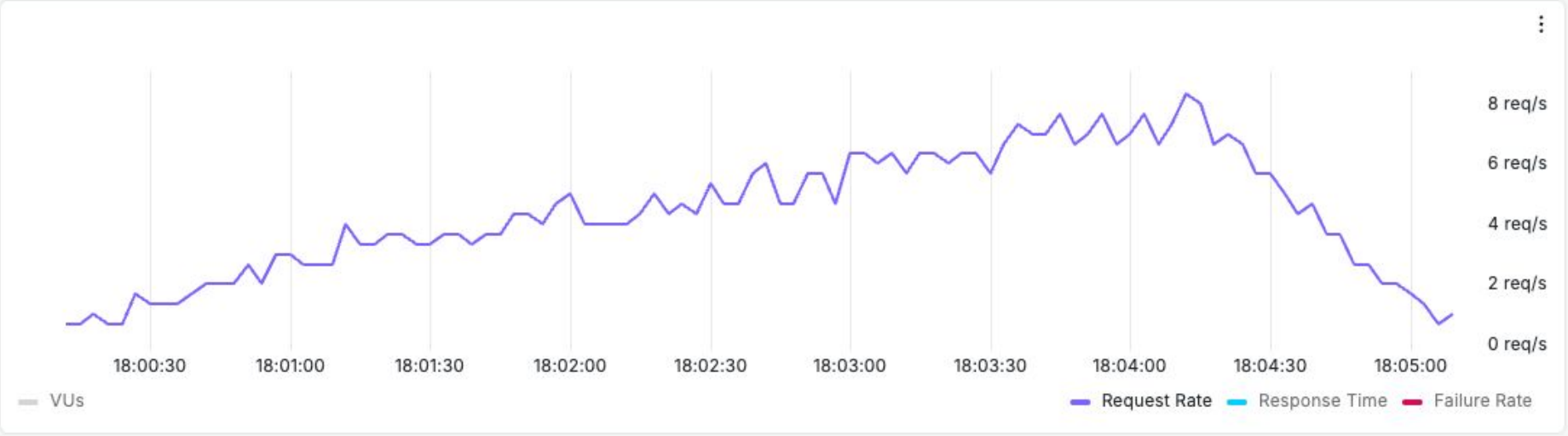
SCENARIOS (0) SHOW

No scenario was configured in this test. Check out how to create one

PERFORMANCE OVERVIEW

The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.

REQUESTS MADE	HTTP FAILURES	PEAK RPS	P95 RESPONSE TIME
1.3K reqs	371 reqs	8.33 reqs/s	477 ms



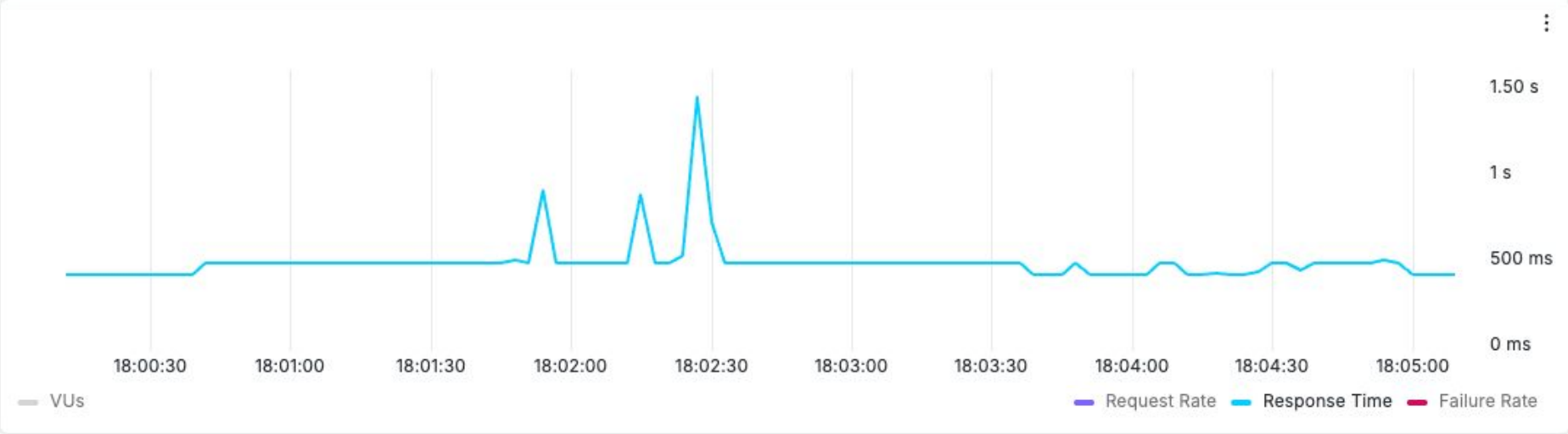
SCENARIOS (0) SHOW

No scenario was configured in this test. Check out how to create one

PERFORMANCE OVERVIEW

The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.

REQUESTS MADE	HTTP FAILURES	PEAK RPS	P95 RESPONSE TIME
1.3K reqs	371 reqs	8.33 reqs/s	477 ms



SCENARIOS (0) SHOW

No scenario was configured in this test. Check out how to create one

PERFORMANCE OVERVIEW

The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.

REQUESTS MADE	HTTP FAILURES	PEAK RPS	P95 RESPONSE TIME
1.3K reqs	371 reqs	8.33 reqs/s	477 ms

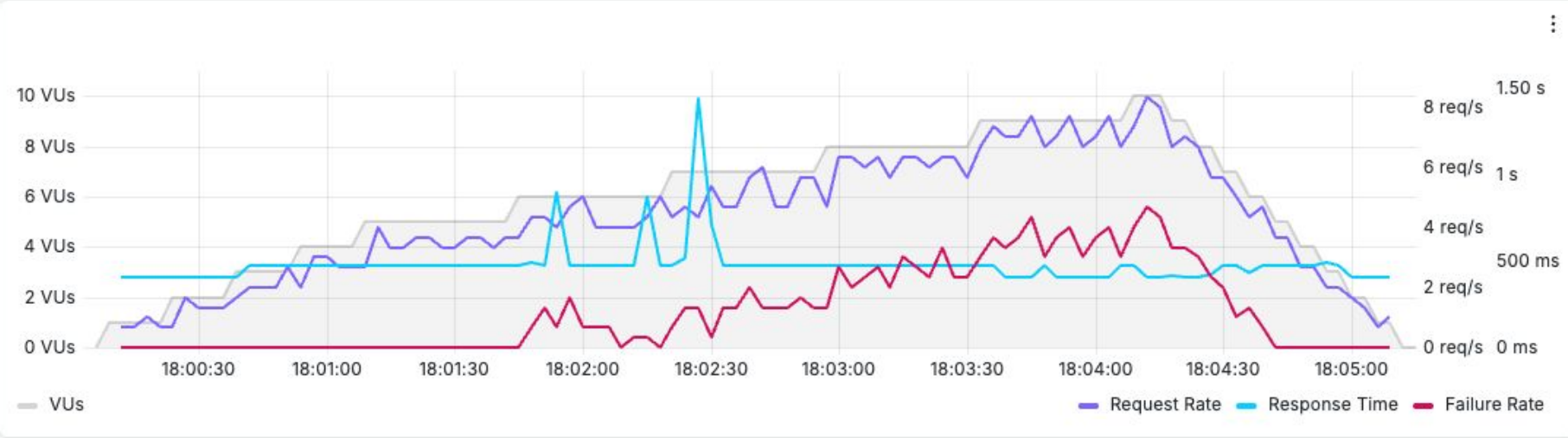
SCENARIOS (0) SHOW

No scenario was configured in this test. Check out how to create one

PERFORMANCE OVERVIEW

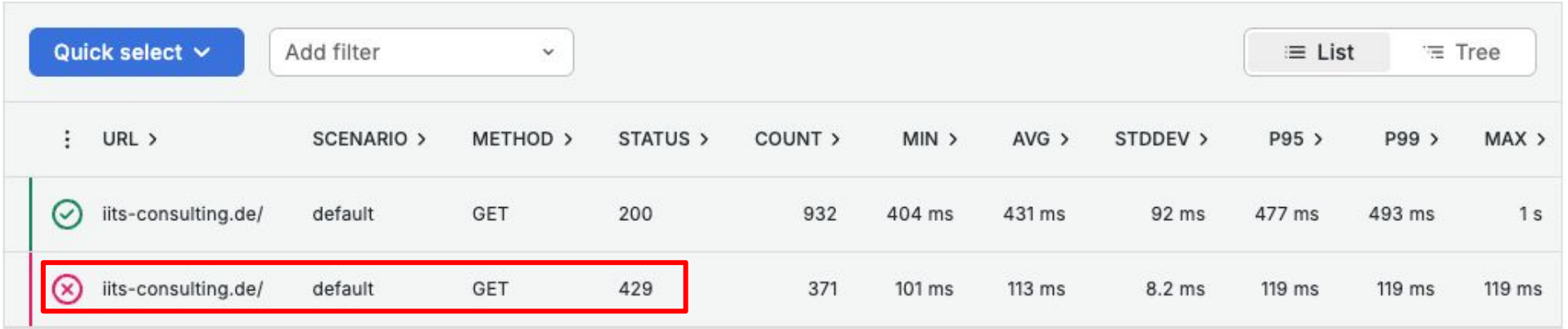
The 95th percentile response time of the system being tested was 477 ms, and 1 303 requests were made with 371 failures at an average rate of 4.2 requests/second.

REQUESTS MADE	HTTP FAILURES	PEAK RPS	P95 RESPONSE TIME
1.3K reqs	371 reqs	8.33 reqs/s	477 ms



SCENARIOS (0) SHOW

No scenario was configured in this test. Check out how to create one



HTTP/1.1 429 Too Many Requests

Gelernte Lektionen (1/3)

Zugriff auf Observability Systeme notwendig:

- Monitoring
- Metriken
- Traces
- Logs

Gelernte Lektionen (2/3)

It’s still hardware (somewhere)

Sometimes 2 Cores < 1 Core (Node types matter)

I/O Caching

Leave some free RAM on the node for I/O Caching

Self-Managed Kubernetes

If you manage Kubernetes yourself: Set kubeReserved and systemReserved before any testing

Gelernte Lektionen (3/3)

Kommunikation

Lasttest vorher mit relevanten Abteilungen und Dienstleistern abstimmen.

Fokus behalten

Nie mehr als eine Änderung gleichzeitig vornehmen.

Dokumentation

Jede Veränderung und das dazugehörige Ergebnis dokumentieren.

Reproduzierbarkeit

Tests wiederholen, um zufällige äußere Faktoren auszuschließen.

Root Cause Analysis

Wenn etwas bricht, die Ursache(n) finden, dokumentieren und dann reparieren

Realistische Erwartung

Dinge werden kaputt gehen – Das ist der Weg

Mögen die Lasttests mit Ihnen sein...

...und Ihnen Erkenntnisse bringen.

Vielen Dank

stephan.schwarz@iits-consulting.de